

Architektura a komponenty Android aplikací

Co je Android

- **Android** = operační systém pro mobilní zařízení
- používá se hlavně na:
 - telefonech
 - tabletech
 - chytrých hodinkách
 - televizích
 - automobilech
- je založený na Linux kernelu
- aplikace se obvykle píší v:
 - Kotlinu
 - Javě
- pro vývoj se používá hlavně Android Studio
- Android poskytuje prostředí, ve kterém mohou bezpečně běžet aplikace a využívat hardware zařízení.

Architektura Androidu

- Android má více vrstev. Zjednodušeně:

```
Aplikace
Application Framework
Android Runtime + nativní knihovny
Hardware Abstraction Layer
Linux Kernel
Hardware
```

Každá vrstva má svůj účel.

Linux Kernel

Co je kernel

- **kernel** = jádro operačního systému
- je nejnižší softwarová vrstva Androidu
- komunikuje přímo s hardwarem
- Android používá upravený Linux kernel

Kernel zajišťuje například:

- správu procesů
- správu paměti
- práci se souborovým systémem
- síťovou komunikaci
- ovladače zařízení
- bezpečnostní mechanismy
- správu napájení

Ovladače zařízení

Kernel obsahuje nebo používá ovladače pro hardware, například:

- displej
- dotykovou vrstvu
- kameru
- Wi-Fi
- Bluetooth
- GPS
- senzory
- zvuk

Aplikace s hardwarem většinou nepracují přímo. Používají vyšší vrstvy Androidu.

Hardware Abstraction Layer

- zkratka **HAL**
- vrstva mezi Android systémem a hardwarem
- sjednocuje přístup k různému hardwaru
- díky ní nemusí aplikace řešit konkrétní model zařízení

Příklad:

- aplikace chce použít kameru
- neřeší konkrétní výrobce kamery
- komunikuje přes Android API
- systém přes HAL komunikuje s konkrétním hardwarem

Nativní knihovny

- Android obsahuje nativní knihovny napsané převážně v C/C++
- používají se pro nízkoúrovňové nebo výkonnostně náročné části systému

Příklady:

- práce s grafikou

- práce s databází SQLite
- multimédia
- vykreslování
- šifrování
- webové jádro

Aplikace je většinou nepoužívají přímo, ale přes Android framework.

ART

Co je ART

- **ART** = Android Runtime
- běhové prostředí pro Android aplikace
- spouští kód aplikací
- nahradilo starší Dalvik runtime

Aplikace napsané v Kotlinu nebo Javě se překládají do bytecode, který Android runtime umí spustit.

K čemu ART slouží

ART zajišťuje:

- spouštění aplikací
- správu paměti
- garbage collector
- optimalizaci běhu aplikace
- izolaci aplikací
- práci s třídami a objekty

Zjednodušeně:

```
Kotlin / Java kód → bytecode → ART → běh aplikace
```

Application Framework

- vyšší vrstva Androidu
- poskytuje aplikacím API pro práci se systémem
- aplikace přes něj používají funkce telefonu

Application Framework poskytuje například:

- správu aktivit
- práci s okny

- přístup k souborům
- práci s notifikacemi
- přístup ke kontaktům
- práci s polohou
- práci s kamerou
- správu oprávnění
- komunikaci mezi aplikacemi

Příklady systémových služeb:

- `ActivityManager`
- `PackageManager`
- `NotificationManager`
- `LocationManager`
- `ContentResolver`

Aplikační vrstva

- nejvyšší vrstva Androidu
- zde běží samotné aplikace
- například:
 - telefon
 - zprávy
 - prohlížeč
 - galerie
 - aplikace třetích stran

Každá aplikace běží odděleně od ostatních.

Sandbox

Co je sandbox

- **sandbox** = izolované prostředí pro běh aplikace
- každá Android aplikace běží ve vlastním prostoru
- aplikace nemůže automaticky přistupovat k datům jiných aplikací
- zvyšuje bezpečnost systému

Každá aplikace má:

- vlastní proces
- vlastní uživatelské ID
- vlastní datový prostor
- vlastní oprávnění

Význam sandboxu

Sandbox chrání:

- data uživatele
- data ostatních aplikací
- systém před škodlivou aplikací
- aplikace navzájem mezi sebou

Příklad:

- hra nemůže automaticky číst SMS
- aplikace nemůže automaticky číst kontakty
- aplikace nemůže bez povolení používat kameru

Android aplikace

Android aplikace není jen jeden soubor s jednou funkcí. Skládá se z více komponent.

Hlavní komponenty Android aplikace:

- Activity
- Service
- BroadcastReceiver
- ContentProvider

Tyto komponenty musí být správně definované a mohou spolu komunikovat.

Activity

Co je Activity

- **Activity** = jedna obrazovka aplikace
- představuje část uživatelského rozhraní
- uživatel s ní přímo interaguje

Příklady:

- přihlašovací obrazovka
- hlavní stránka aplikace
- detail produktu
- nastavení aplikace

Jedna aplikace může mít více aktivit.

Příklad:

Úkol Activity

Activity řeší:

- zobrazení UI
- reakce na kliknutí
- zadávání dat uživatelem
- přechody mezi obrazovkami
- práci s životním cyklem obrazovky

V moderních aplikacích se často používá jedna hlavní Activity a více obrazovek se řeší přes navigaci v Compose.

Lifecycle Activity

Co je lifecycle

- **lifecycle** = životní cyklus komponenty
- popisuje, v jakém stavu se Activity nachází
- Android může Activity spouštět, pozastavovat, obnovovat nebo rušit

Důležité metody životního cyklu Activity:

- `onCreate()`
- `onStart()`
- `onResume()`
- `onPause()`
- `onStop()`
- `onDestroy()`

`onCreate()`

- volá se při vytvoření Activity
- obvykle se zde nastavuje UI
- inicializují se základní části obrazovky

`onStart()`

- Activity začíná být viditelná pro uživatele

`onResume()`

- Activity je aktivní
- uživatel s ní může pracovat

onPause()

- Activity přestává být aktivní
- například se otevře jiné okno nebo dialog

onStop()

- Activity už není viditelná

onDestroy()

- Activity se ruší
- může nastat při ukončení nebo zničení systémem

Zjednodušeně:

```
onCreate → onStart → onResume → onPause → onStop → onDestroy
```

Service

Co je Service

- **Service** = komponenta pro práci na pozadí
- nemá vlastní uživatelské rozhraní
- používá se pro dlouhodobější nebo neviditelné úlohy

Příklady použití:

- přehrávání hudby
- stahování dat
- synchronizace
- sledování polohy
- komunikace se serverem

Typy služeb

Foreground Service

- běží na pozadí, ale uživatel o ní ví
- musí zobrazovat notifikaci
- používá se pro důležité úlohy

Příklad:

- přehrávání hudby
- navigace

- sportovní aplikace měřící trasu

Background Service

- běží na pozadí bez přímé interakce uživatele
- moderní Android její použití výrazně omezuje kvůli baterii a výkonu

Bound Service

- jiná komponenta se ke službě připojí
- používá se, když Activity potřebuje se službou komunikovat

Broadcast Receiver

Co je Broadcast Receiver

- **BroadcastReceiver** = komponenta, která přijímá systémové nebo aplikační zprávy
- reaguje na události, které nastanou v systému nebo aplikaci

Příklady událostí:

- změna připojení k síti
- zapnutí zařízení
- připojení nabíječky
- nízký stav baterie
- příchozí systémová událost
- vlastní zpráva z aplikace

Jak funguje

- systém vyšle broadcast
- aplikace s příslušným receiverem ho může přijmout
- receiver provede krátkou reakci

Příklad:

```
System: změnila se síť  
BroadcastReceiver: aplikace na to zareaguje
```

Poznámka

BroadcastReceiver by neměl dělat dlouhou práci. Pokud je potřeba něco delšího, obvykle se spustí služba nebo naplánovaná úloha.

Content Provider

Co je Content Provider

- **ContentProvider** = komponenta pro sdílení dat mezi aplikacemi
- umožňuje jedné aplikaci bezpečně poskytovat data jiné aplikaci
- data jsou dostupná přes jednotné rozhraní

Příklady dat:

- kontakty
- kalendář
- média
- soubory
- vlastní data aplikace

Jak se používá

Aplikace k datům přistupuje přes `ContentResolver`.

Příklad:

```
Aplikace → ContentResolver → ContentProvider → data
```

Význam

ContentProvider umožňuje:

- sdílet data mezi aplikacemi
- řídit přístup k datům
- používat jednotné API
- chránit data pomocí oprávnění

Příklad:

- aplikace chce vybrat fotku z galerie
- použije systémové rozhraní
- data získá bezpečně přes systém

Android Manifest

Co je AndroidManifest.xml

- **AndroidManifest.xml** = povinný konfigurační soubor Android aplikace
- obsahuje důležité informace o aplikaci
- systém ho čte při instalaci a spouštění aplikace

Manifest definuje například:

- název aplikace
- balíček aplikace
- hlavní Activity
- komponenty aplikace
- oprávnění
- podporované vlastnosti zařízení
- intent filtry
- minimální verzi SDK

Co se zapisuje do Manifestu

Komponenty aplikace

V manifestu mohou být zapsány:

- Activity
- Service
- BroadcastReceiver
- ContentProvider

Příklad:

```
<activity android:name=".MainActivity" />
```

Oprávnění

Aplikace musí v manifestu uvést, jaká oprávnění chce používat.

Příklad:

```
<uses-permission android:name="android.permission.CAMERA" />
```

Intent filter

Intent filter říká, na jaké intenty může komponenta reagovat.

Příklad:

```
<intent-filter>  
    <action android:name="android.intent.action.MAIN" />  
    <category android:name="android.intent.category.LAUNCHER" />  
</intent-filter>
```

Význam:

- Activity je spouštěcí obrazovka aplikace
- zobrazí se v launcheru

Oprávnění v Androidu

Co jsou oprávnění

- oprávnění určují, k jakým citlivým funkcím nebo datům může aplikace přistupovat
- chrání soukromí a bezpečnost uživatele

Příklady oprávnění:

- kamera
- mikrofon
- poloha
- kontakty
- kalendář
- úložiště
- Bluetooth
- notifikace

Oprávnění v manifestu

Některá oprávnění musí být zapsána v manifestu.

Příklad:

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

Runtime permissions

- některá oprávnění nestačí jen zapsat do manifestu
- aplikace se na ně musí zeptat uživatele za běhu
- říká se tomu **runtime permissions**

Příklad:

- aplikace chce použít kameru
- systém zobrazí dialog
- uživatel povolí nebo zamítne přístup

Proč se práva řeší za běhu

- uživatel má větší kontrolu
- aplikace nedostane citlivá práva automaticky

- snižuje se riziko zneužití

Intent

Co je Intent

- **Intent** = zpráva nebo požadavek na provedení akce
- používá se pro komunikaci mezi komponentami
- může spustit Activity, Service nebo předat data

Intent může říkat:

- co chci udělat
- s jakými daty
- jaká komponenta to má zpracovat

Explicitní Intent

- určuje konkrétní komponentu, která se má spustit
- používá se hlavně uvnitř vlastní aplikace

Příklad:

```
MainActivity → DetailActivity
```

Význam:

- aplikace přesně ví, jakou Activity chce otevřít

Implicitní Intent

- neurčuje konkrétní komponentu
- říká jen, jakou akci chce provést
- systém najde aplikace, které akci umí zpracovat

Příklad:

```
Otevřít webovou stránku  
Sdílet text  
Vyfotit fotku  
Vybrat soubor
```

Pokud je více možností, systém se může zeptat uživatele, jakou aplikaci chce použít.

Příklad:

- otevření odkazu
- uživatel si vybere prohlížeč

Intent filter

- říká, jaké intenty umí komponenta přijímat
- zapisuje se v manifestu
- používá se hlavně pro implicitní intenty

Příklad:

- aplikace deklaruje, že umí otevřít určitý typ souboru
- systém ji nabídne jako možnost

Jetpack Compose

Co je Jetpack Compose

- **Jetpack Compose** = moderní framework pro tvorbu UI v Androidu
- používá deklarativní přístup
- UI se popisuje pomocí funkcí
- je doporučovaný pro moderní Android vývoj
- používá se hlavně s Kotlinem

Deklarativní UI

- vývojář popisuje, jak má UI vypadat pro daný stav
- když se stav změní, UI se překreslí podle nového stavu
- neřeší se tolik ruční upravování jednotlivých prvků

Zjednodušeně:

```
stav aplikace → UI
```

Příklad:

```
@Composable
fun Greeting(name: String) {
    Text("Ahoj $name")
}
```

Výhody Compose

- méně XML
- kratší zápis UI

- lepší práce se stavem
- snadnější tvorba komponent
- dobrá integrace s Kotlinem
- moderní přístup k UI

MVVM

Co je MVVM

- **MVVM** = Model-View-ViewModel
- architektonický vzor pro rozdělení aplikace
- pomáhá oddělit UI od logiky aplikace
- často se používá v Android aplikacích

Části MVVM

Model

- představuje data a business logiku
- pracuje například s:
 - databází
 - API
 - repository
 - datovými třídami

Příklad:

Student, Produkt, Objednavka

View

- uživatelské rozhraní
- zobrazuje data uživateli
- předává akce uživatele dál

V Androidu může být View:

- Activity
- Fragment
- Compose obrazovka

ViewModel

- prostředník mezi View a Modelem

- uchovává stav obrazovky
- připravuje data pro zobrazení
- přežívá některé změny konfigurace, například otočení obrazovky
- neměl by přímo obsahovat UI prvky

Zjednodušeně:

```
View → ViewModel → Model  
View ← ViewModel ← Model
```

Výhody MVVM

- lepší oddělení zodpovědností
- přehlednější kód
- jednodušší testování
- ViewModel drží stav obrazovky
- UI může pouze reagovat na změny stavu

Lifecycle v moderní aplikaci

- Android komponenty mají životní cyklus
- systém může komponenty vytvářet, pozastavovat nebo rušit
- aplikace musí s těmito změnami počítat

Důvody změn lifecycle:

- uživatel odejde z aplikace
- přijde hovor
- otočí se obrazovka
- systém potřebuje paměť
- otevře se jiná Activity

Lifecycle a ViewModel

- ViewModel pomáhá uchovat stav obrazovky
- při změně konfigurace se Activity může znovu vytvořit
- ViewModel může přežít tuto změnu
- díky tomu se neztratí data obrazovky

Příklad:

- uživatel vyplňuje formulář
- otočí telefon
- Activity se obnoví

- ViewModel může držet rozepsaná data

Lifecycle a Compose

- Compose UI se překresluje podle stavu
- změna stavu způsobí recomposition
- vývojář by měl oddělit stav od samotného UI
- často se používá ViewModel jako zdroj stavu

Recomposition

- **recomposition** = znovuvykreslení části UI při změně stavu
- Compose překreslí jen to, co je potřeba
- umožňuje deklarativní způsob práce

Příklad:

změní se počet položek → Compose aktualizuje seznam

Shrnutí komponent

Komponenta	Význam
Activity	obrazovka aplikace
Service	práce na pozadí
BroadcastReceiver	reakce na systémové nebo aplikační události
ContentProvider	sdílení dat mezi aplikacemi
Manifest	konfigurace aplikace
Intent	zpráva / požadavek na akci
Permission	oprávnění k citlivým funkcím
Sandbox	izolované prostředí aplikace
ART	běhové prostředí Android aplikací
Compose	moderní deklarativní UI framework
MVVM	architektonický vzor
Lifecycle	životní cyklus komponenty